Vector Space Model and Ranked Retrieval on the Cranfield Collection

Information Retrieval — Programming Assignment II

1. Group details

Group name: **search_ninjas**. All submitted files carry this prefix.

- Krish Charniya

2. Problem statement

Build a ranked retrieval pipeline over the Cranfield collection (1,400 aerodynamics abstracts, 225 test queries, 1,612 graded relevance judgements) using an open-source search engine, and tune the preprocessing, the weighting model and its parameters to maximise retrieval effectiveness. Only sparse vector space models are permitted: no advanced probabilistic models, and no dense or neural retrieval. Indexing and search times are to be reported alongside effectiveness, and the resulting system is to be evaluated on further unseen queries.

3. Implementation details

The system is built on **PyTerrier 1.1.2** driving **Terrier 5.11**. Terrier supplies the inverted index, the term pipeline and the weighting models; our code supplies the collection parsing, the experiment plan and the evaluation harness.

3.1 Programs

File	Role
`search_ninjas_setup.sh`	creates the virtualenv, fetches a JDK, unpacks the collection
`search_ninjas_env.py`	points JAVA_HOME at the bundled JDK before pyjnius loads the JVM
`search_ninjas_parse.py`	readers for cran.all.1400, cran.qry and cranqrel
`search_ninjas_index.py`	builds a Terrier index under one preprocessing variant, timed
`search_ninjas_experiments.py`	the three-stage experiment plan; writes every table in this report
`search_ninjas_search.py`	runs the tuned pipeline over a query file, writes a TREC run
`search_ninjas_report.py`	renders this PDF from the experiment output

3.2 Preprocessing

The PA1 preprocessing steps are repeated, but as Terrier's term pipeline rather than our own code: case folding and non-alphanumeric stripping in the English tokeniser, stopword removal against the list supplied with PA1, and Porter stemming. Each step is switchable so that Stage 1 can measure what it contributes. The same normalisation is applied to query text before it reaches Terrier's parser, so query terms and index terms cannot drift apart.

3.3 Two properties of the collection that the code has to get right

Query numbering. `cranqrel` numbers the queries 1–225 by their *position* in `cran.qry`, whereas `cran.qry`'s own `.I` labels are non-contiguous (001, 002, 004, 008, … 365). Keying a run on the labels and scoring it against `cranqrel` silently evaluates almost every query against the wrong judgements. Our run files are written under both numberings.

Relevance grades. Cleverdon's codes run 1 (a complete answer) to 4 (minimum interest) — smaller is better, the reverse of what every evaluation tool assumes. They are inverted to gains 4–1 before use; the 225 rows coded `-1` are unjudged markers and are dropped.

Empty records. Documents 471 and 995 have no title, author, bibliography or abstract at all. Document 995 is judged relevant for query 125, so it is an unreachable relevant document that places a hard ceiling on recall.

3.4 Parsing

A Cranfield record is written `.T .A .B .W`. The document parser is a state machine keyed on the most recent field tag with one extra rule: once `.W` has opened, an `.A` or `.B` line is stray text inside the abstract rather than a new field. Document 240 contains exactly this, and without the rule it loses roughly fifteen lines of its abstract.

4. Plan of experiments

Three stages, each one narrowing the next: **(1)** preprocessing, meaning which fields, stemmer and stopword list to index; **(2)** weighting model at default parameters; **(3)** the parameters of the winning model.

> Stage 3 tunes on the **odd-numbered** queries (113 of them) and reports on the held-out **even-numbered** queries (112), as well as on the full set. The interleaved split is deliberate: the Cranfield queries are grouped by subject, so a contiguous split would tune on a different subject mix than it reports on. Tuning happens only in Stage 3, so the held-out half is the sole estimate here of what the unseen test queries will produce, and every figure labelled *held-out* below comes from queries the tuning never saw.

Effectiveness is measured with `ir_measures`: MAP (the primary metric used for all selection), nDCG@10 and nDCG@20 on the graded judgements, P@5, P@10, recall@100 and reciprocal rank. Improvements over a baseline are accompanied by a paired sign test; the `+` and `-` columns count the queries that improved and degraded.

5. Results

5.1 Stage 1 — preprocessing

Twenty-seven indices: three field sets × three stemmers × three stopword settings, each scored with untuned BM25.

Configuration	AP	nDCG@10	P@10	R@100	terms	postings	avg len	index (s)	search (ms/q)
all__porter__assignment	0.3185	0.3763	0.2382	0.7376	6,495	92,349	106.24	0.68	6.59
title_text__porter__assignment	0.3156	0.3729	0.2378	0.7397	4,603	80,630	97.36	0.65	6.62
all__porter__terrier	0.3132	0.3706	0.2396	0.7426	6,452	89,178	103.80	0.64	6.92
title_text__porter__terrier	0.3116	0.3680	0.2391	0.7446	4,561	81,049	97.86	0.66	6.89
all__weakporter__assignment	0.3021	0.3586	0.2360	0.7303	7,518	94,232	106.24	0.62	6.53
text__porter__terrier	0.3006	0.3575	0.2347	0.7445	4,561	81,049	89.88	0.36	6.35
text__porter__assignment	0.3005	0.3602	0.2342	0.7366	4,603	80,630	89.43	0.37	6.26
title_text__weakporter__assignment	0.2995	0.3552	0.2351	0.7302	5,601	82,512	97.36	0.36	5.96
all__weakporter__terrier	0.2978	0.3518	0.2329	0.7371	7,465	91,036	103.80	0.42	6.10
title_text__weakporter__terrier	0.2963	0.3504	0.2329	0.7383	5,549	82,906	97.86	0.37	6.03
all__none__assignment	0.2904	0.3538	0.2324	0.7101	9,128	98,273	106.24	0.69	6.22
all__none__terrier	0.2895	0.3538	0.2342	0.7112	9,085	95,066	103.80	0.68	6.52
title_text__none__terrier	0.2887	0.3527	0.2329	0.7122	7,147	86,930	97.86	0.39	5.41
title_text__none__assignment	0.2887	0.3526	0.2302	0.7124	7,189	86,548	97.36	0.39	5.26
text__weakporter__assignment	0.2882	0.3455	0.2329	0.7262	5,601	82,512	89.43	0.36	6.02
text__weakporter__terrier	0.2854	0.3400	0.2298	0.7358	5,549	82,906	89.88	0.39	6.10
text__none__assignment	0.2796	0.3439	0.2262	0.7088	7,189	86,547	89.43	0.40	5.36
text__none__terrier	0.2772	0.3405	0.2271	0.7106	7,147	86,929	89.88	0.44	5.68
title_text__porter__none	0.2079	0.2414	0.1591	0.6640	4,815	116,502	173.81	0.61	8.27
all__porter__none	0.2054	0.2424	0.1613	0.6600	6,705	128,354	183.46	0.67	8.33
all__weakporter__none	0.2010	0.2374	0.1613	0.6471	7,739	130,514	183.46	0.66	8.56
title_text__weakporter__none	0.2003	0.2337	0.1582	0.6514	5,822	118,643	173.81	0.41	7.70
all__none__none	0.1945	0.2300	0.1529	0.6215	9,400	134,796	183.46	0.67	8.74
title_text__none__none	0.1927	0.2283	0.1556	0.6224	7,458	122,918	173.81	0.39	7.74
text__porter__none	0.1926	0.2243	0.1511	0.6494	4,815	116,502	161.90	0.39	7.79
text__weakporter__none	0.1864	0.2154	0.1471	0.6423	5,822	118,643	161.90	0.39	7.77
text__none__none	0.1795	0.2110	0.1440	0.6121	7,458	122,917	161.90	0.95	8.64

Table 1: all 27 preprocessing variants, ordered by MAP. Names read `fields__stemmer__stopwords`.

5.2 Stage 2 — weighting models

All models at their default parameters, on the Stage 1 winning index.

Configuration	AP	nDCG@10	nDCG@20	P@5	P@10	R@100	RR	search (ms/q)
TF_IDF	0.3203	0.3778	0.4197	0.3307	0.2373	0.7470	0.5569	6.43
BM25	0.3185	0.3763	0.4192	0.3316	0.2382	0.7376	0.5498	6.28
LemurTF_IDF	0.2953	0.3543	0.3914	0.2969	0.2284	0.7265	0.5045	6.32
Tf	0.1927	0.2352	0.2767	0.1964	0.1507	0.6516	0.4302	7.32

Table 2: weighting models at default parameters over the full 225 queries.

5.3 Stage 3 — parameter tuning

110 BM25 settings (k_1 × b), scored on the training half. Tuned against default on the held-out half:

Configuration	AP	nDCG@10	nDCG@20	P@5	P@10	R@100	RR	better	worse	p
BM25(k_1=1.2, b=0.75) [default]	0.3031	0.3731	0.4079	0.3268	0.2375	0.7252	0.5432			
BM25(k_1=2.5, b=0.75) [tuned]	0.3069	0.3771	0.4149	0.3232	0.2402	0.7366	0.5446	67	37	0.378595

Table 3: effect of parameter tuning on the 112 held-out queries.

Configuration	AP	nDCG@10	nDCG@20	P@5	P@10	R@100	RR	better	worse	p
BM25(k_1=1.2, b=0.75) [default]	0.3185	0.3763	0.4192	0.3316	0.2382	0.7376	0.5498			
BM25(k_1=2.5, b=0.75) [tuned]	0.3227	0.3838	0.4252	0.3333	0.2471	0.7520	0.5499	130	76	0.281116

Table 4: the same comparison over all 225 queries.

5.4 Final configuration and timings

Setting	Value
Indexed fields	all
Stemmer	porter
Stopword list	assignment
Weighting model	BM25 (bm25.k_1 = 2.5, bm25.b = 0.75)
Indexing time (1,400 documents)	0.99 s
Search time	6.2 ms / query
MAP, held-out queries	0.3069
MAP, all 225 queries	0.3227

Table 5: the submitted configuration. Indexing is a single-threaded in-memory build; search time is wall clock over all 225 queries divided by 225.

6. Discussion of results

6.1 Preprocessing dominates everything else

The single largest effect in the whole study is stopword removal. The best variant with no stopword list reaches MAP 0.2079; the best with one reaches 0.3185, a gain of +53.2%, larger than every model and parameter decision combined. Cranfield queries are full English questions ("what similarity laws must be obeyed when…"), so without a stopword list the query vector is dominated by terms that match almost every document. Porter stemming is the second largest effect, worth roughly 0.03 MAP, and switching from Porter to Terrier's weak Porter loses most of that — on 1,400 short abstracts, aggressive conflation is the right trade.

Field choice barely matters. Adding the author and bibliography fields to the title and abstract is worth about 0.003 MAP, and indexing the abstract alone rather than title plus abstract costs about 0.015 — small because in most Cranfield records the title is repeated as the opening of the abstract, so the title field is nearly redundant already.

6.2 The IDF component carries the weighting

Raw term frequency with no IDF component (MAP 0.1927) is well under two thirds as effective as anything that weights by term rarity, which confirms that the vector space model's IDF component is doing most of the work. Among the properly weighted models the spread is narrow: TF–IDF reaches 0.3203 and BM25 0.3185, while LemurTF–IDF's different length normalisation costs about 0.025.

Parameter tuning then adds remarkably little. On the held-out queries the tuned model gains +0.0038 MAP over its own default, improving 67 queries and degrading 37 for a sign-test p-value of 0.379, which is not significant. The response surface is shallow rather than flat: MAP varies by 0.027 across the settings between $k_1 = 0.8$ and 4.0 and $b = 0.3$ and 1.0, and by 0.049 across the full grid. The choice is therefore not free, but it is small next to the preprocessing effect, and the gap between the best and the default setting is smaller than the noise between two halves of the query set.

One methodological note: an earlier, narrower sweep put the optimal k_1 on the boundary of the grid, which indicates the grid is too small rather than giving an answer. The grid reported above extends to $k_1 = 4.0$ so that the optimum is interior.

6.3 Models excluded by the assignment

The assignment restricts the system to sparse vector space models and rules out advanced probabilistic and dense neural retrieval. Two families that Terrier offers are therefore excluded even though they are available at no extra cost: the Divergence-From-Randomness weighting models, and pseudo-relevance feedback, which rewrites the query from the terms of the top-ranked documents rather than scoring the query as given. Both are probabilistic rather than vector space methods. The submitted system is consequently tuned BM25 over the Stage 1 index, with no query rewriting.

6.4 Where the ceiling is

End to end, the pipeline moves MAP from 0.3185 (untuned BM25 on the winning index) to 0.3227 over the full query set, +1.3%. Recall@100 finishes at 0.7520, so roughly a fifth of the relevant documents are still missed in the top 100, and at least one of them, document 995 for query 125, is unreachable by any lexical system because the record is empty. The remaining gap is largely vocabulary mismatch between the query wording and the abstract wording, which term weighting alone cannot bridge.

6.5 Threats to validity

The held-out half is 112 queries, so a MAP difference below roughly 0.01 is not distinguishable from noise; we have reported sign tests rather than relying on the point estimates. Stage 1 selected the index on the full query set before the train/test split was introduced in Stage 3, so the preprocessing choice is mildly optimistic, though the effect is large enough and stable enough across models that the conclusion is not in doubt.